



MACHINE LEARNING BASED ALGORITHM SELECTION FOR SORTING NUMERIC ARRAYS

Presented by

AHMED SALAH ABDALHI MOHAMMED

Supervised by

Assoc. Prof. Dr. Emre Özbilge

Research Objective

$S : f(x) \rightarrow A$

reduce expected regret with respect to the oracle

01 compact structural array features

informative for algorithm selection and
inexpensive to extract

02 offline XGBoost model

train and evaluate per-instance algorithm
selection

03 leakage-resistant evaluation

quantify static selection under rigorous
protocol

04 reproducible Python library

package the complete method for deployment
and further research

Prediction is made before any candidate algorithm is executed

The selector extracts features once, runs one XGBoost prediction, and chooses one portfolio member.

Allowed before sorting

Extract $O(n)$ features

Measure order, runs, repetition, distribution, robust scale

Predict introsort, heapsort, or timsort

Not allowed at prediction time

Sort with introsort

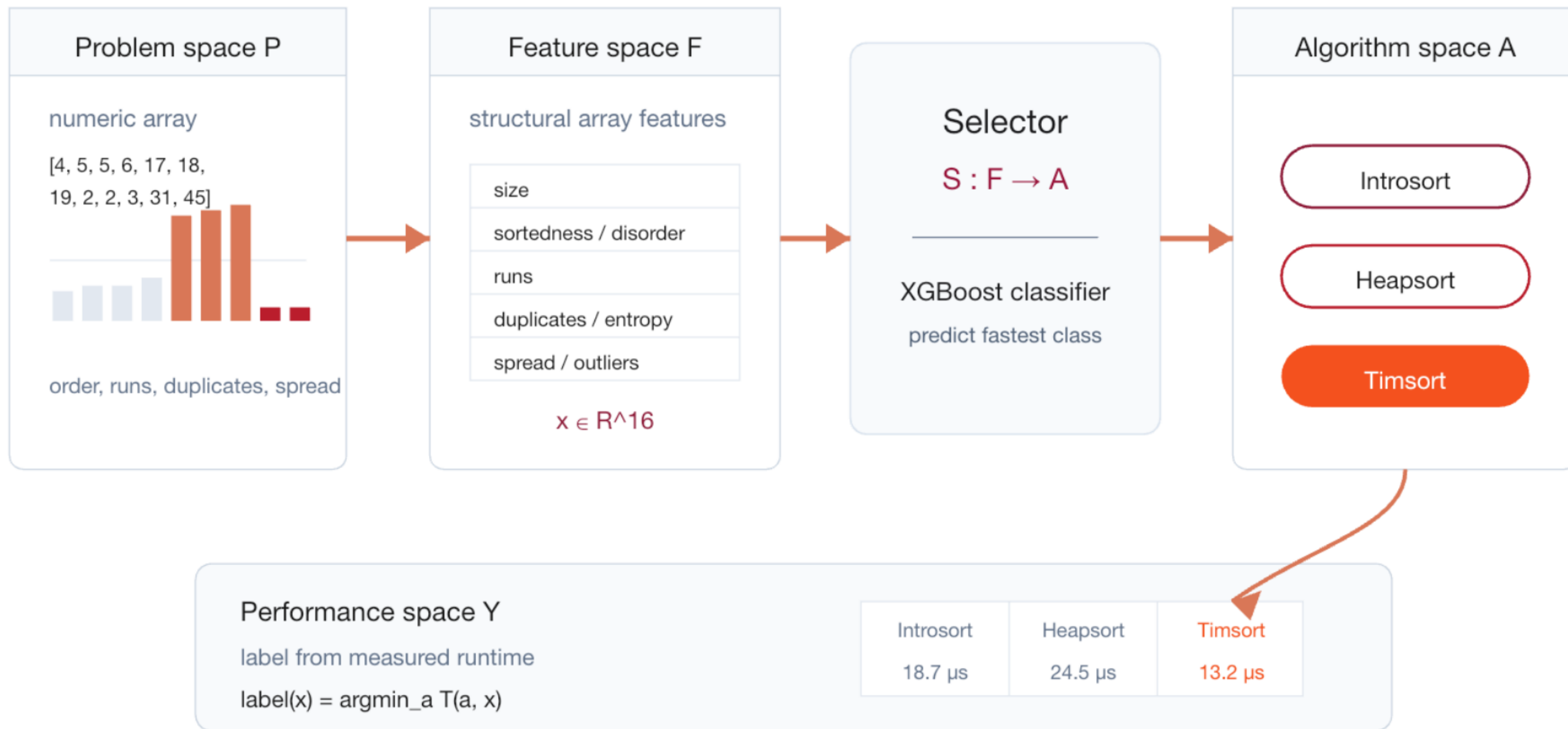
Sort with heapsort

Sort with timsort

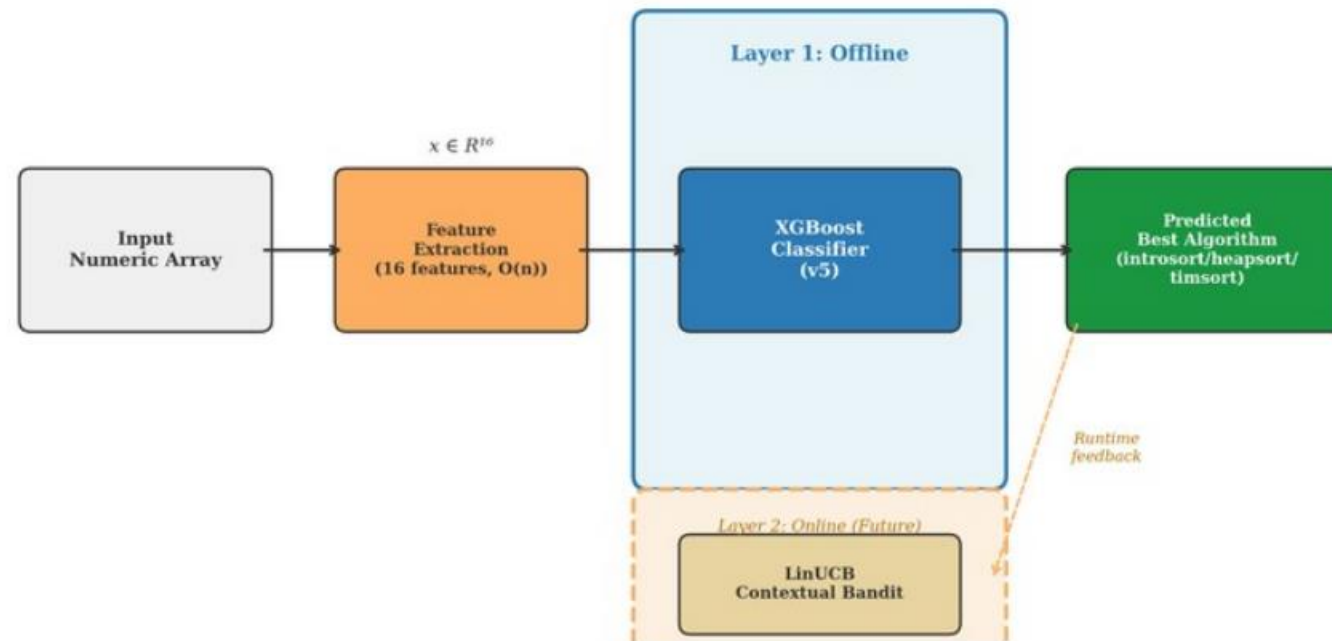
Pick after seeing all times

The oracle is used only for evaluation; it is not available at run time.

Algorithm Selection Problem



System Architecture



Main track

all domains · 3 algorithms

Prediction path

features → XGBoost classifier

Separate track

F1-specific routing stays separate from the main result

The corpus uses five structurally diverse real-world domains

1,188,265 numeric arrays were extracted to maximise structural diversity.

1.18M

labeled array instances

Formula 1 telemetry

Equity prices

Cryptocurrency OHLCV

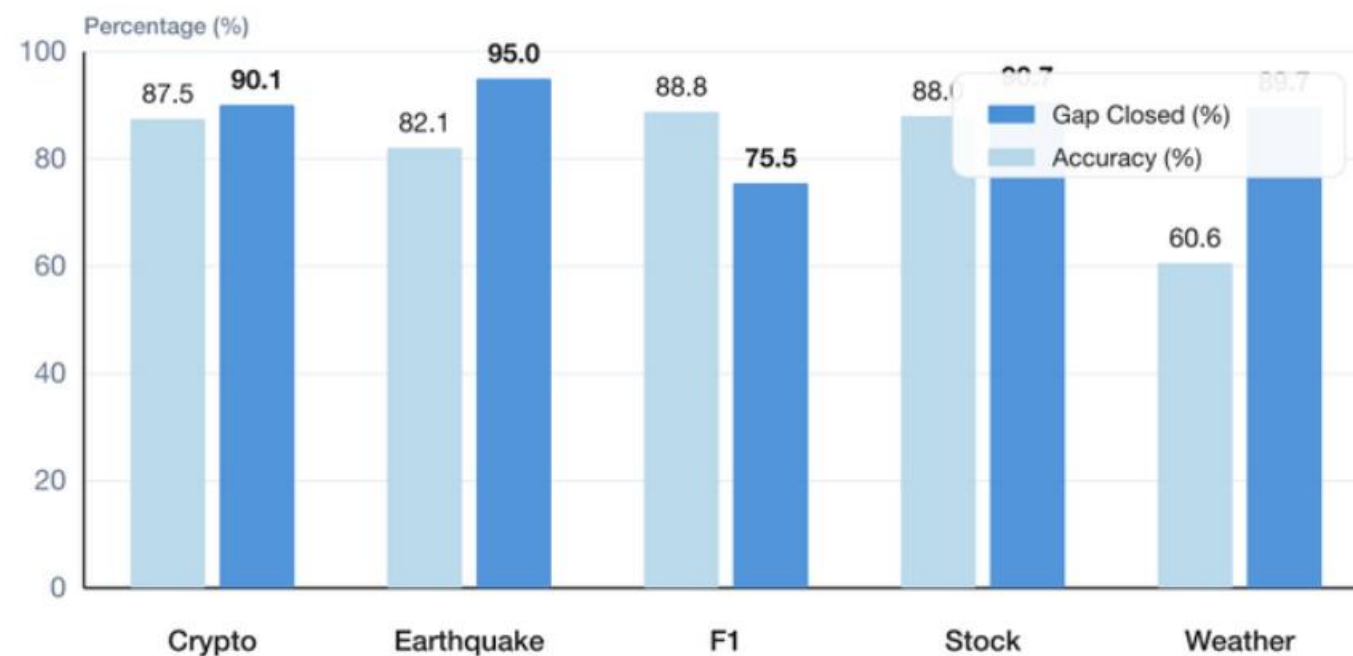
Weather

Seismic measurements

Real data was preferred because synthetic distributions failed to converge to genuine structure.

Leave-one-domain-out testing supports cross-domain robustness

Each fold trains on four domains and tests on the unseen fifth domain.



Five real domains, five different numeric traces

Stock market

n = 6,835

XLB · max · rolling_5d_mean



min 4.92
med 14.5
max 53.22

[5.873, 5.919, 5.968, 5.974, 6.021, ...]

stock_XLB_max_1d_rolling_5d_mean.csv

Cryptocurrency

n = 2,500

XLM-USD · 1mo · momentum_5d_w2500c0



min -0.0083
med -0.0001
max 0.0108

[0.0009, 0.0005, 0.0002, -0.0003, ...]

crypto_XLM-USD_1mo_5m_momentum_5d_w2500c0.csv

Weather

n = 184,081

Nairobi · 2000_2020 · wind_rolling_24h_std



min 0.5445
med 3.259
max 8.55

[3.389, 3.385, 3.341, 3.269, 3.164, ...]

weather_Nairobi_wind_rolling_24h_std.csv

Formula 1 telemetry

n = 286

Canadian Grand Prix · R · VER · lap 68 · Z



min 0.0797
med 2025
max 4329

[0.0797, 12.79, 28.74, 51.06, 79.76, 108.46, ...]

f1_2024_R9_R_VER_68.0_Z.csv

Seismic events

n = 200

california · 20200101_20200630 · latitude_c200_1



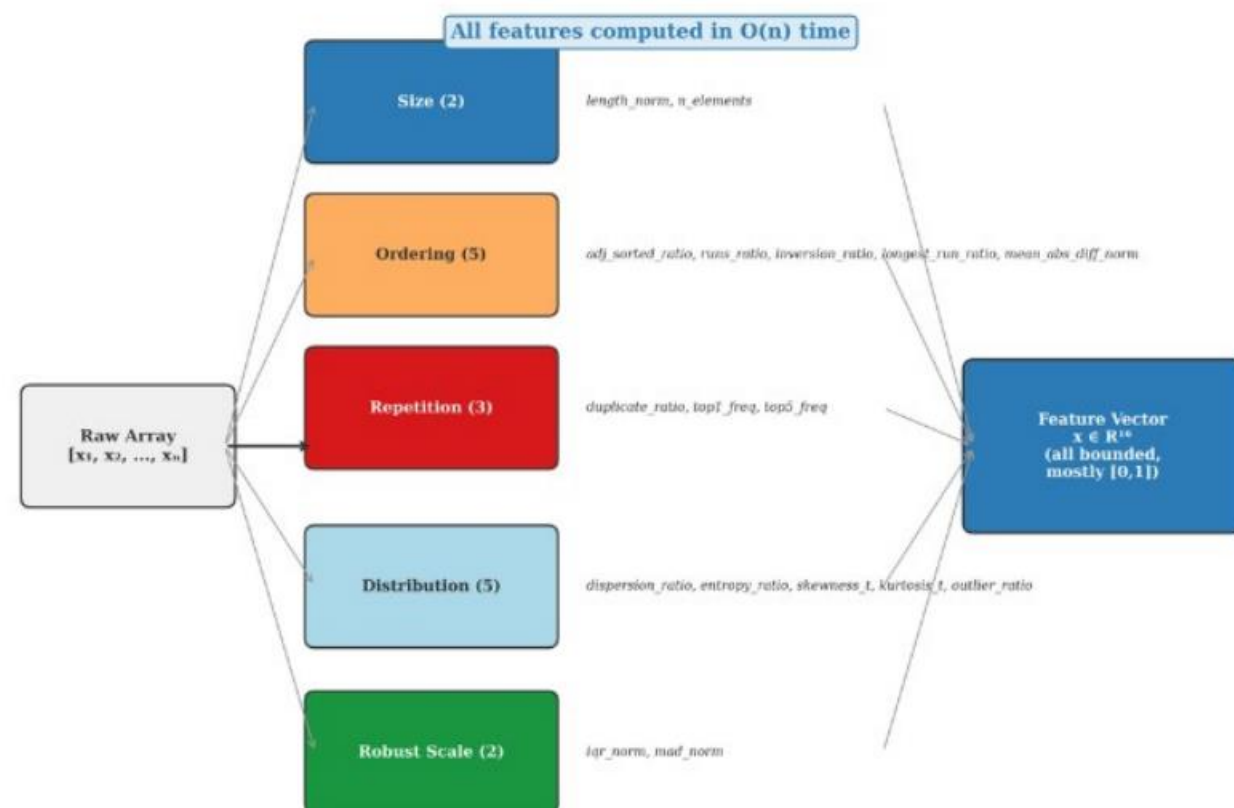
min 32.39
med 35.91
max 40.91

[37.83, 34.02, 36.10, 35.95, 33.54, 36.80, ...]

quake_california_Mlplus_latitude_c200_1.csv

Sixteen $O(n)$ structural features characterise each array

Every feature must be cheap to compute and linked to differential sorting behaviour.



Size

length_norm
size effects
cache / constants

Ordering

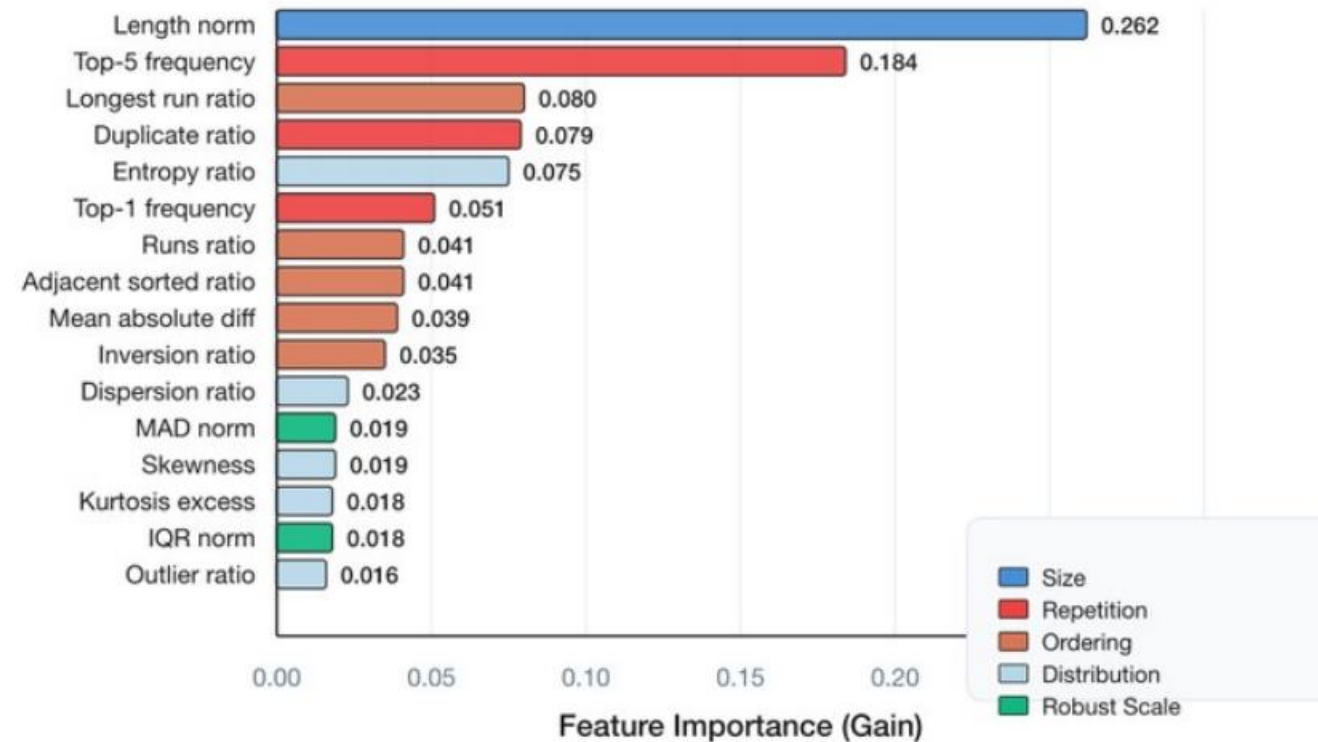
adj_sorted_ratio
runs_ratio
inversion_ratio

Timing features are excluded at prediction time.

A single canonical feature extractor is shared across data generation, training, and inference.

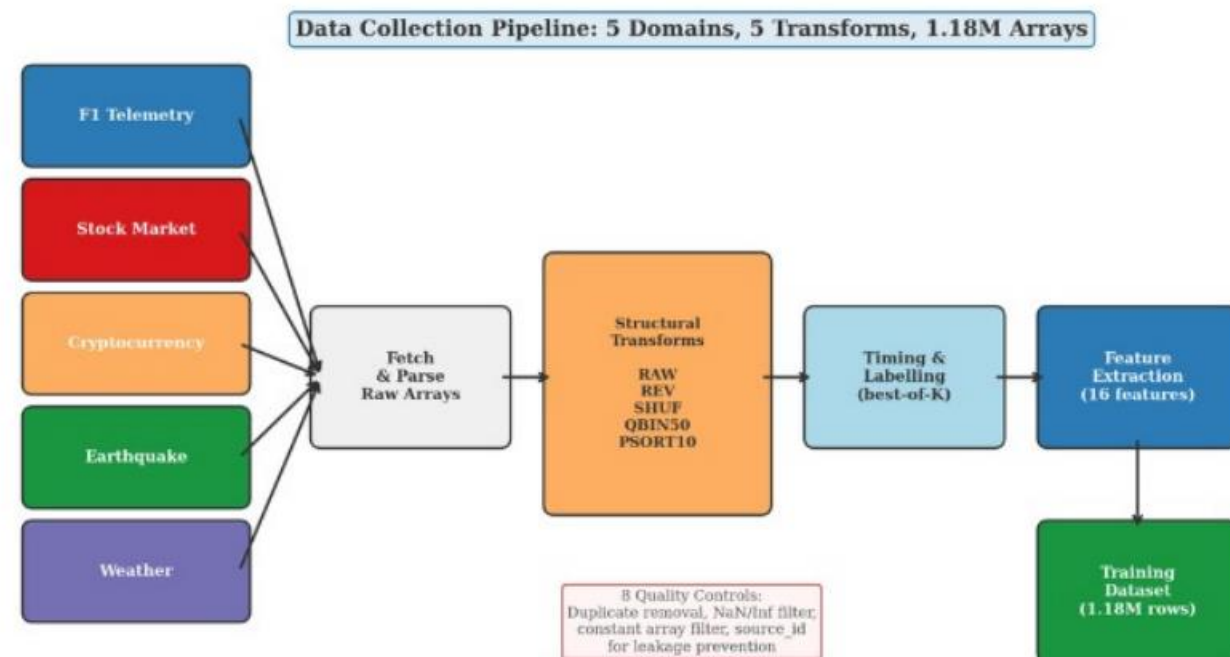
Feature importance follows structural sorting signals

Length, repetition, and ordering account for most of the discriminative signal.



Labels are empirical fastest-algorithm measurements

Each candidate algorithm is timed on a fresh copy; the minimum observed runtime becomes the label.



RAW

REV

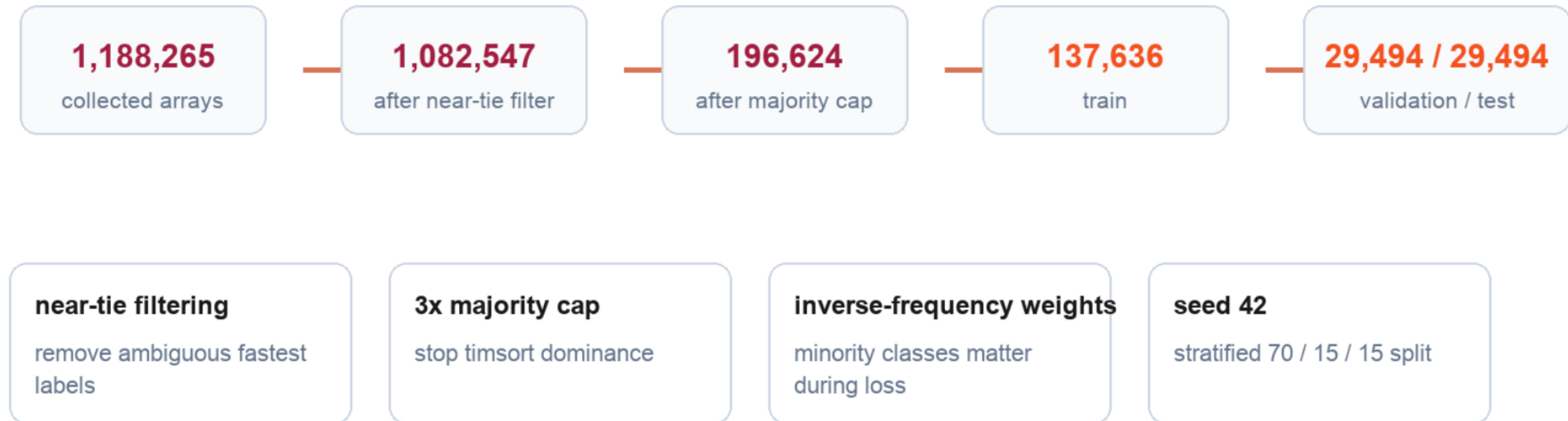
SHUF

QBIN50

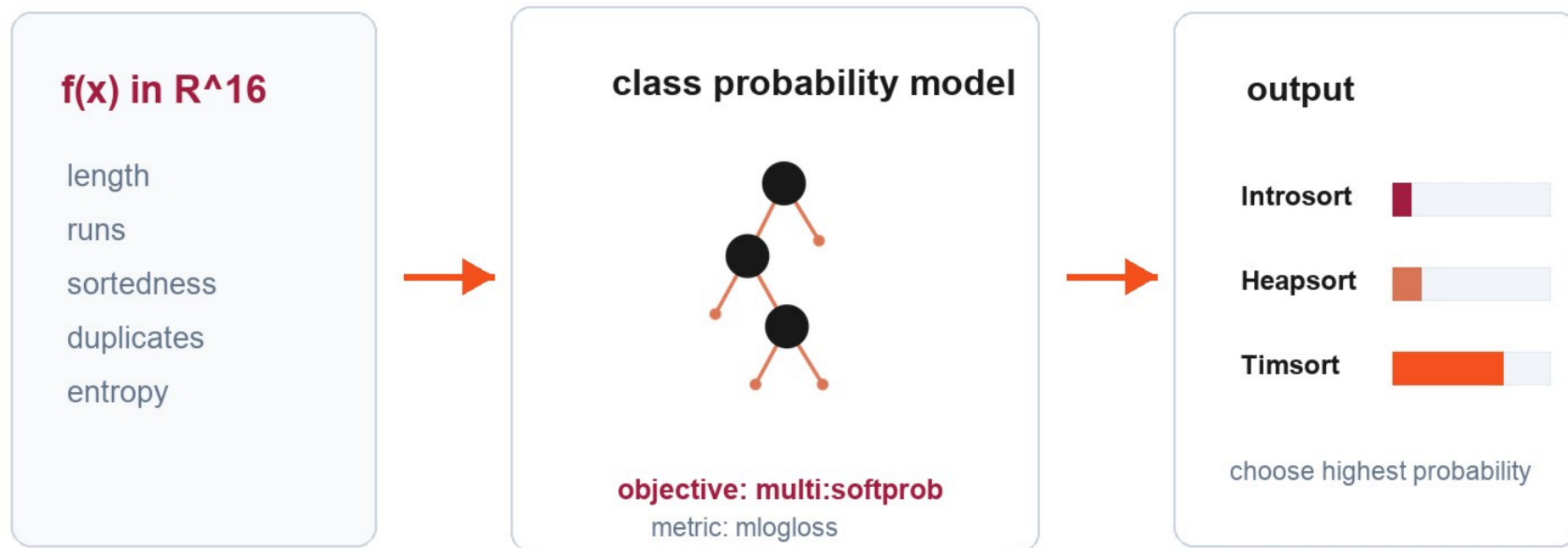
PSORT10

RAW, REV, SHUF, QBIN50, and PSORT10 force a non-trivial decision boundary.

Training Dataset and Split



The model learns a probability for each sorting choice



The training signal is balanced before the trees learn



near-tie filtering + majority cap + inverse-frequency sample weights

XGBoost corrects class scores using first and second order information

$$g_{ik} = \partial_{\hat{y}_{ik}} \ell(y_i, \hat{p}_i)$$

first-order gradient

direction of score correction



$$h_{ik} = \partial_{\hat{y}_{ik}}^2 \ell(y_i, \hat{p}_i)$$

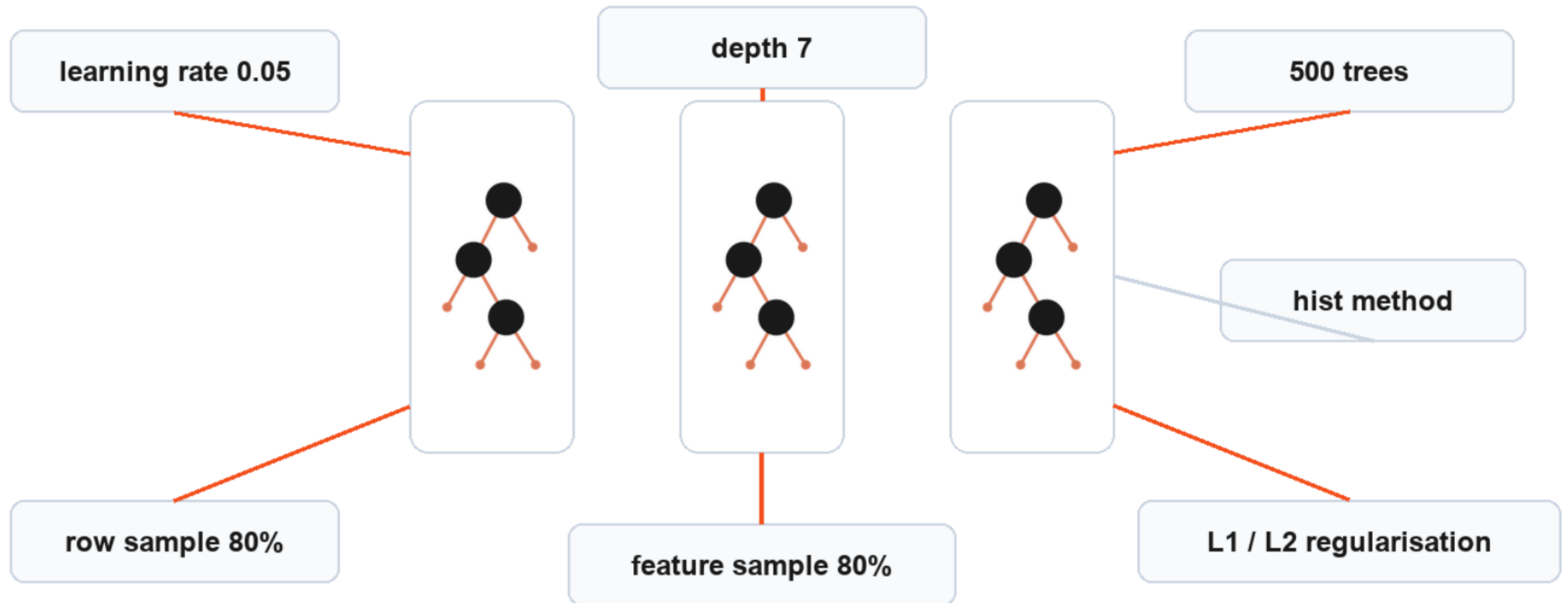
second-order Hessian

curvature / strength of correction

multi:softprob + mlogloss

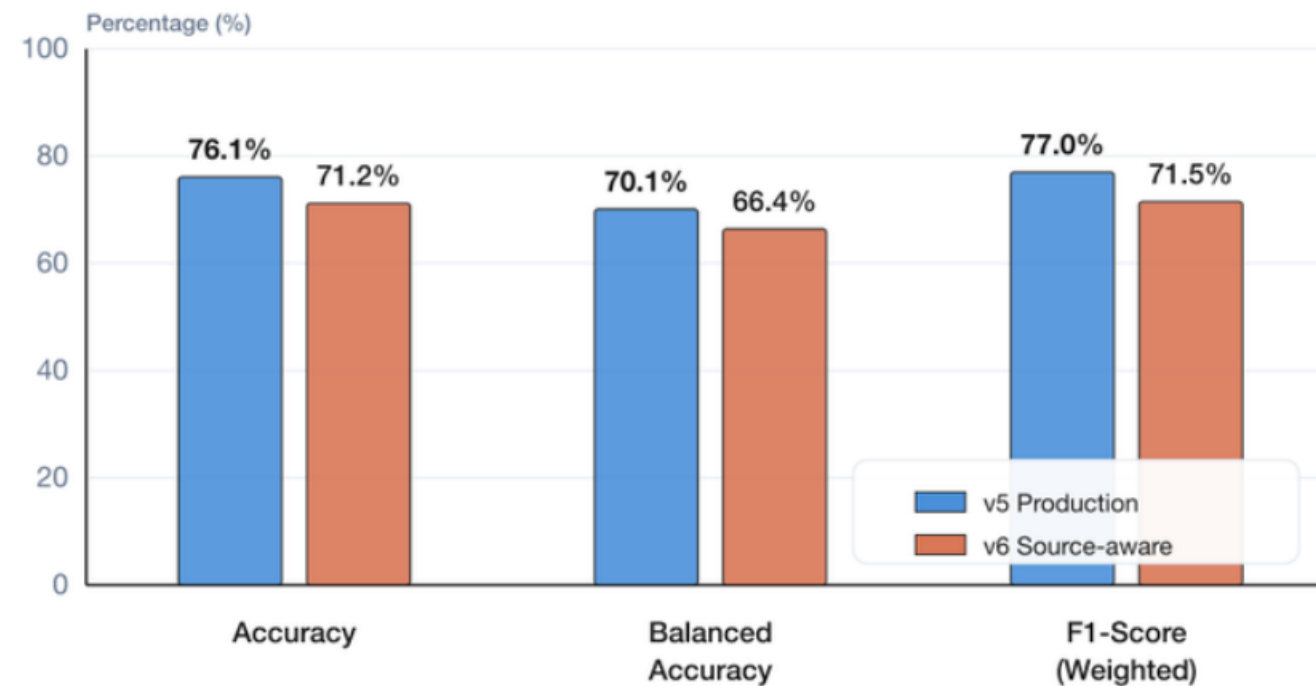
XGBoost uses both terms when choosing splits and leaf corrections

The ensemble is constrained so it does not overfit timing noise



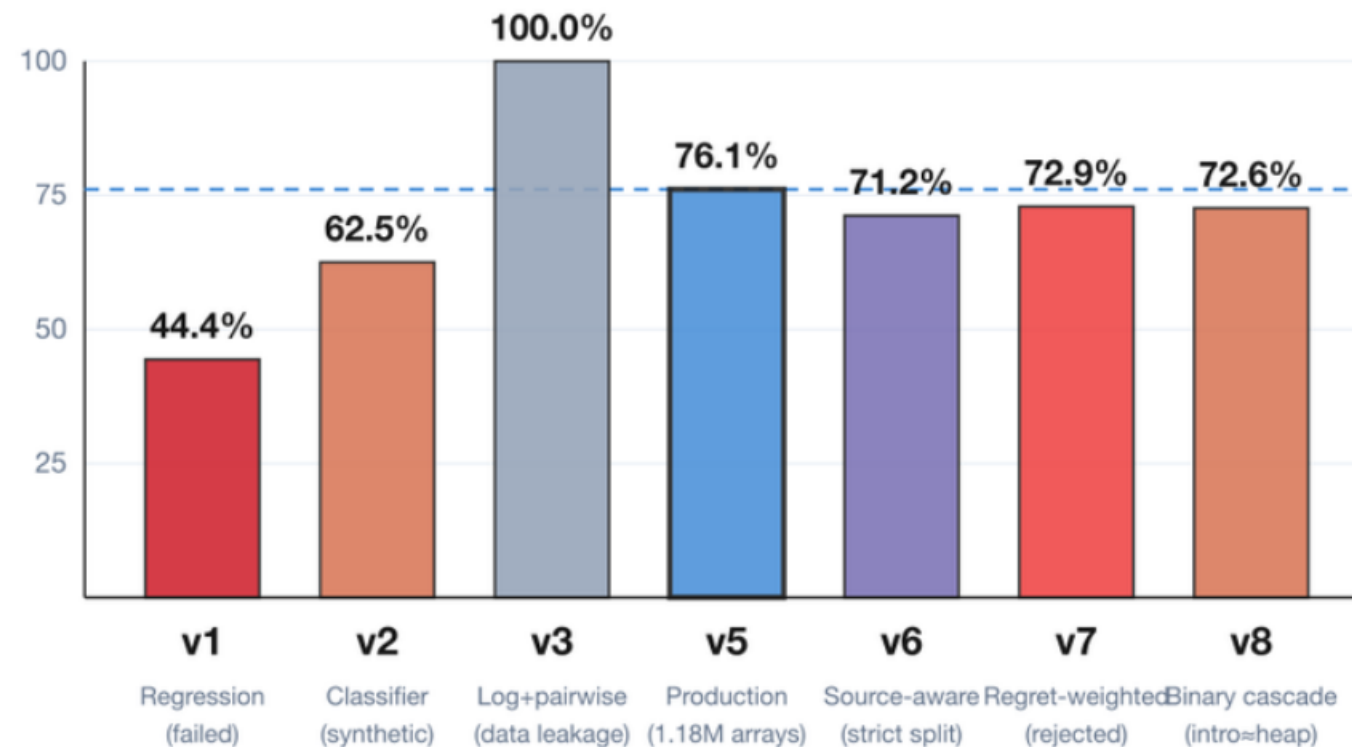
Strict checks define the boundary of the reported result

The reported result stays with v5; later variants explain leakage risk, objective risk, and the hard boundary.



Model evolution reduced the problem to its deployable form

Regression, timing features, source-aware checks, regret-aware loss, and cascade tests define why v5 is retained



Evaluation measures distance from SBS to VBS

Accuracy is reported, but selector quality is judged by regret and gap closed.

SBS

best fixed algorithm
over the evaluation set

VBS

oracle fastest algorithm
for each array

Model

choose before execution
using structural features

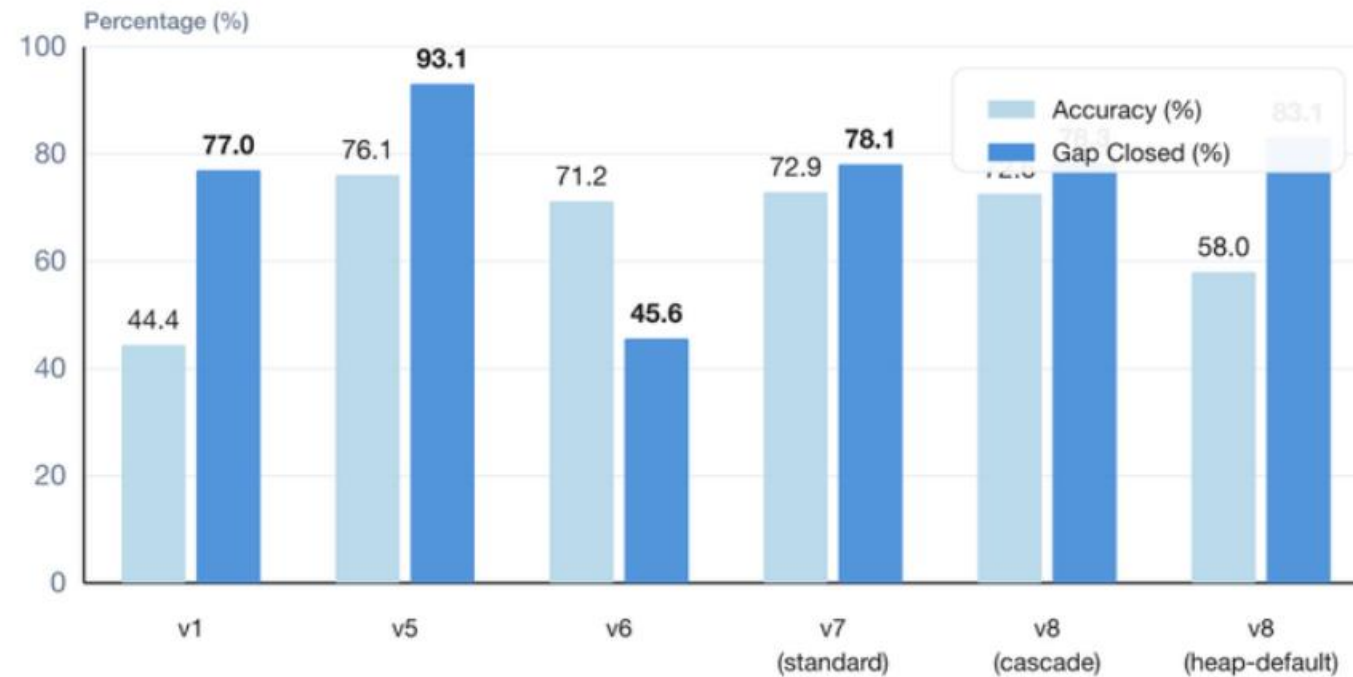
$$\text{Regret} = \frac{T_{\text{model}} - T_{\text{VBS}}}{T_{\text{VBS}}}$$

$$\text{Gap Closed} = \frac{T_{\text{SBS}} - T_{\text{model}}}{T_{\text{SBS}} - T_{\text{VBS}}}$$

Gap closed gives the fraction of the SBS-to-VBS improvement captured by the selector.

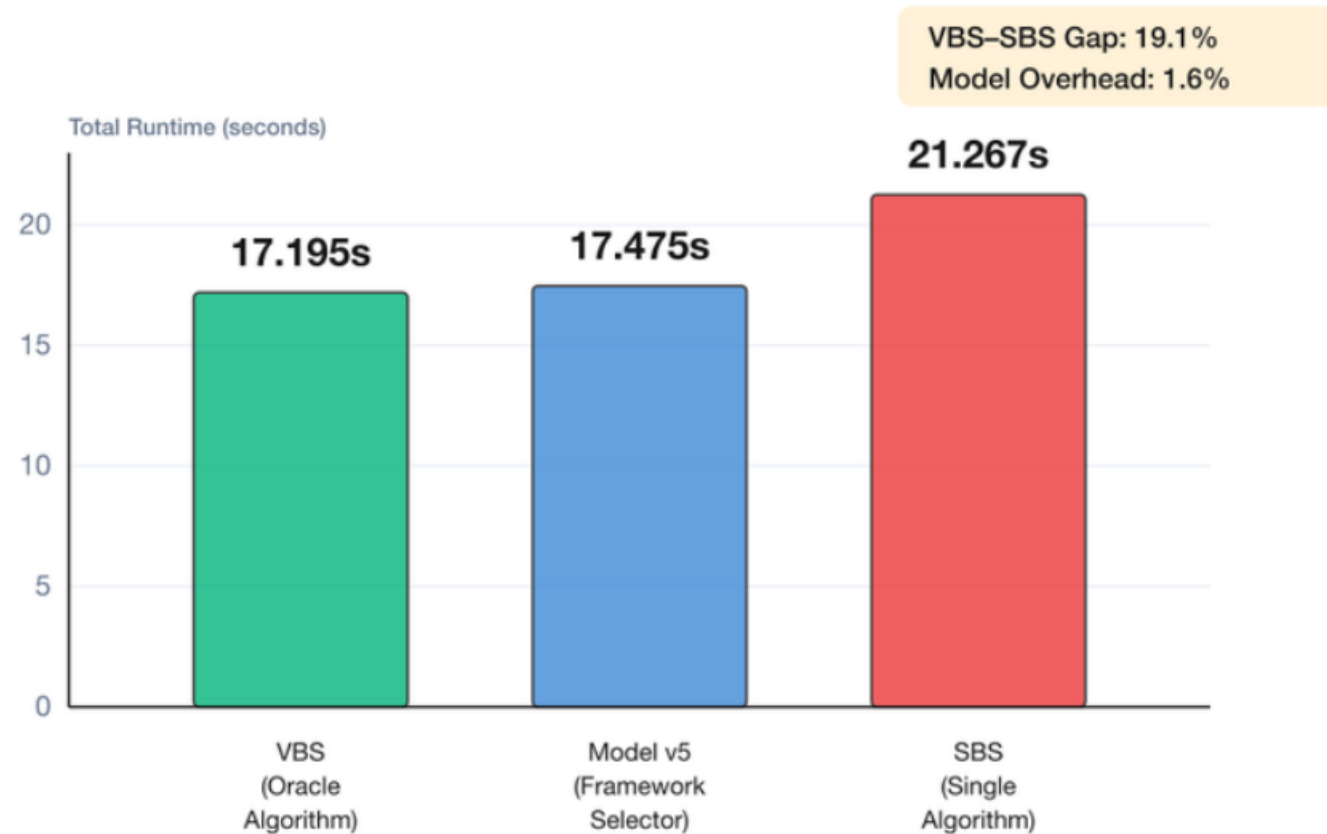
Accuracy alone is insufficient for algorithm selection

A wrong label can be harmless when the selected and optimal algorithms have nearly equal runtimes.



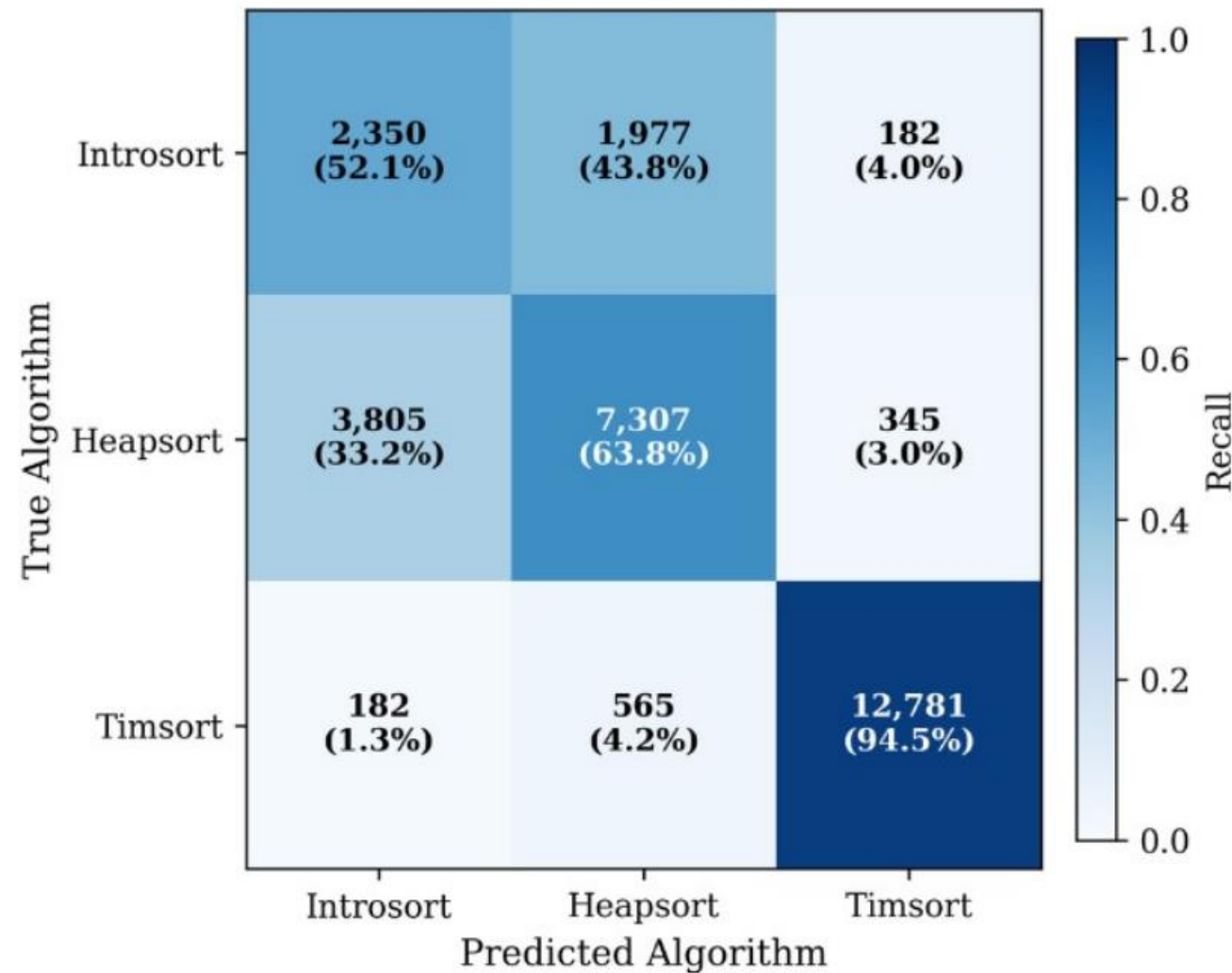
The v5 selector closes most of the SBS-to-VBS gap

The reported result is 76.1% top-1 accuracy, 93.1% gap closed, and 89.6% zero regret.



Residual error is concentrated at the introsort-heapsort boundary

Timsort has a strong structural trace; introsort and heapsort often depend on lower-level effects.



94.5%

timsort recall

Interpretation

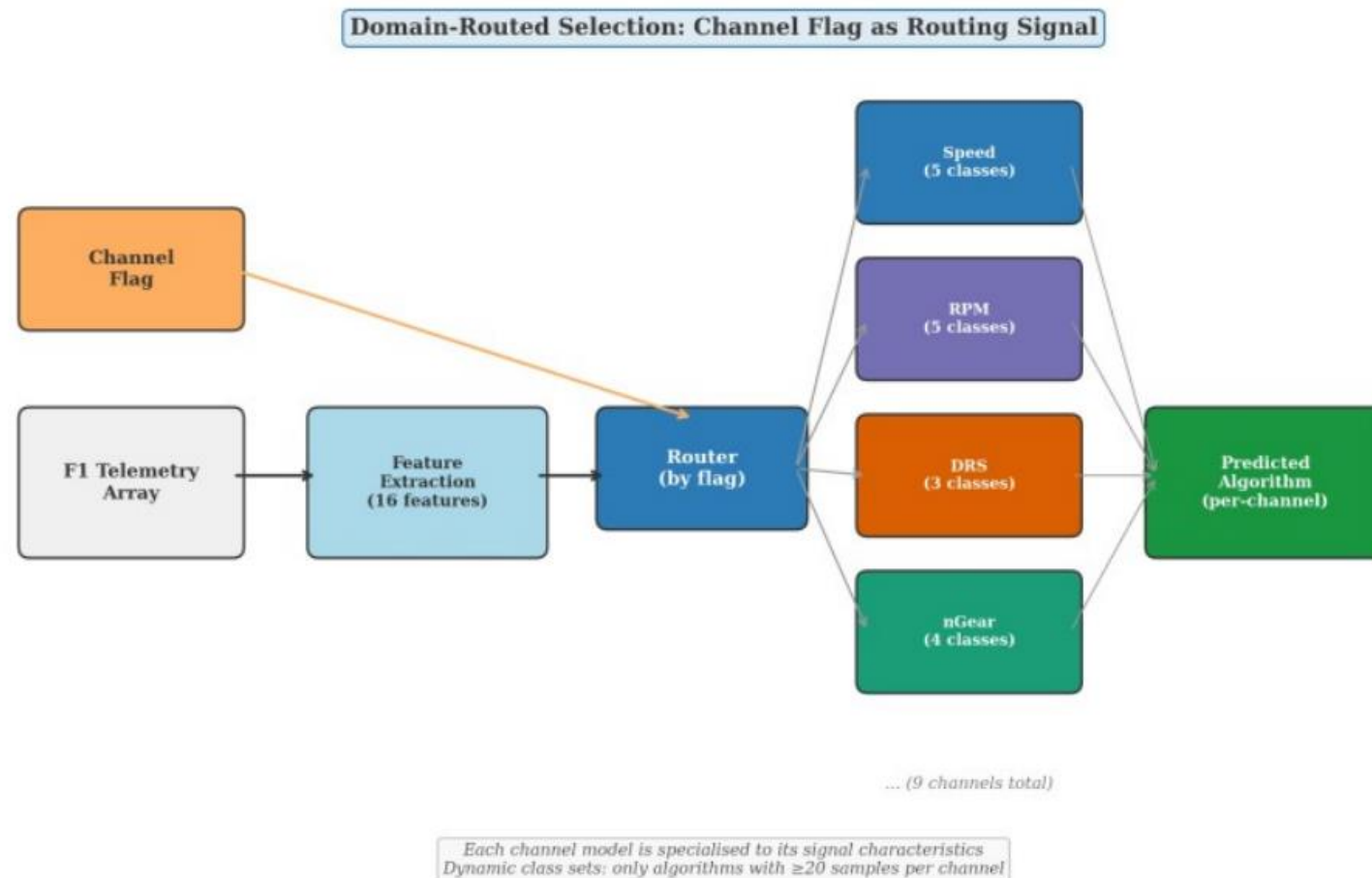
timsort is separable when runs/order are visible.

introsort and heapsort remain close in the current feature space.

Most residual boundary errors carry low measured regret.

F1 routing was treated as a separate specialization

The main result remains v5; the F1 flag/channel route explains the hardest domain.



Main track

v5 general selector

all domains · 3 algorithms

F1 specialization

channel flag routes F1 arrays
to specialised channel models.

flag / channel route

Claim boundary

This does not replace v5.
It stays separate from the
main cross-domain result.

The contribution is a selector and an evaluation discipline

The contributions are the selector, the evaluation discipline, and the reproducible artefact.

01 A deployable selector

A compact pipeline combines sixteen $O(n)$ structural features with a gradient-boosted classifier.

02 A rigorous evaluation framework

Source-level separation, near-tie filtering, LODO testing, and regret metrics give a stronger view than accuracy alone.

03 A reproducible artefact

One source-of-truth feature extractor, data pipeline, and exported XGBoost JSON / ONNX models support replication.

The remaining errors show where the features stop seeing

The limits are portfolio scope, single hardware setting, unvalidated online adaptation, and small-array overhead.

Limitations

- 01 portfolio limited to three C-level comparison sorts
- 02 absolute regret tied to one hardware/software environment
- 03 LinUCB contextual bandit designed but not validated
- 04 feature extraction becomes unprofitable below about 1,000 elements

Future work

- 05 larger portfolios: Powersort, radix/bucket guards, SIMD/GPU sorts
- 06 validated LinUCB under gradual and abrupt distribution shift
- 07 domain embeddings and stricter channel-level routing evaluation
- 08 cross-platform deployment with feature-extraction overhead

Future work extends the portfolio, adaptation, and generalisation axes while preserving cost-aware evaluation.

Thank you
Questions?